

ISTITUTO TECNICO MARCONI DALMINE

Indirizzo Informatica

# Sviluppo di un Videogioco Multiplayer in Tempo Reale

*Architettura Ibrida TCP/UDP per Giochi Web-Based*

**Studenti:**

Diego Ripamonti

Jerome Colcol

Francesco Daminelli

**Classe:** 5<sup>a</sup> Ci

**A.S.:** 2025/2026

22 maggio 2026

# Indice

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                      | <b>2</b> |
| <b>2</b> | <b>Analisi del Problema</b>              | <b>2</b> |
| 2.1      | Soluzione Adottata . . . . .             | 2        |
| <b>3</b> | <b>Architettura del Sistema</b>          | <b>2</b> |
| <b>4</b> | <b>Progettazione del Database</b>        | <b>2</b> |
| 4.1      | Schema Relazionale . . . . .             | 3        |
| <b>5</b> | <b>Protocollo di Comunicazione</b>       | <b>3</b> |
| 5.1      | Handshake e Join . . . . .               | 3        |
| 5.2      | Snapshot del Mondo (UDP) . . . . .       | 3        |
| <b>6</b> | <b>Sviluppo Tecnico</b>                  | <b>3</b> |
| 6.1      | Logica del Server . . . . .              | 3        |
| 6.2      | Intelligenza Artificiale (Bot) . . . . . | 4        |
| 6.3      | Interpolazione Client-Side . . . . .     | 4        |
| <b>7</b> | <b>Conclusioni</b>                       | <b>4</b> |

# 1 Introduzione

Il progetto consiste nella realizzazione di un videogioco multiplayer *real-time* ispirato al celebre *Agar.io*. L'obiettivo didattico principale è l'analisi e l'implementazione di una comunicazione di rete efficiente, affrontando le problematiche di latenza e sincronizzazione tipiche dei sistemi distribuiti ad alta frequenza di aggiornamento.

A differenza delle applicazioni web tradizionali basate esclusivamente su protocollo HTTP o WebSocket, questo progetto esplora l'utilizzo combinato di **TCP** e **UDP** per ottimizzare il throughput e l'affidabilità dei dati trasmessi.

## 2 Analisi del Problema

Nei giochi multiplayer frenetici, il "lag" (latenza) rappresenta il problema principale. Un'architettura basata solo su TCP garantisce l'ordine e la consegna dei pacchetti, ma introduce ritardi significativi a causa del meccanismo di ritrasmissione in caso di perdita di dati (*Head-of-Line Blocking*). D'altra parte, UDP è estremamente veloce ma non garantisce né l'ordine né la consegna dei pacchetti.

### 2.1 Soluzione Adottata

La soluzione implementata è un'architettura ibrida:

- **TCP**: Utilizzato per messaggi critici che richiedono certezza della consegna (login, chat, eventi di morte, classifica).
- **UDP**: Utilizzato per lo scambio continuo dello stato del mondo (posizioni, dimensioni) dove la perdita di un singolo aggiornamento è trascurabile rispetto all'esigenza di riceverne subito uno più recente.

## 3 Architettura del Sistema

Il sistema è suddiviso in tre macro-componenti:

1. **Game Server (Node.js)**: Gestisce la logica di gioco, la fisica, i bot e i socket TCP/UDP.
2. **Client Bridge (Node.js)**: Poiché i browser non supportano socket TCP/UDP grezzi (*raw*), questo componente funge da intermediario tra le WebSocket del browser e i socket di rete del server.
3. **Frontend (HTML5/Canvas)**: Interfaccia utente che renderizza il gioco a 60 FPS utilizzando le API Canvas.

## 4 Progettazione del Database

Per la gestione degli utenti e della persistenza dei punteggi, è stato progettato un database relazionale (MySQL).

## 4.1 Schema Relazionale

Lo schema si compone di due tabelle principali per separare i dati sensibili dalle statistiche di gioco:

- **Credenziali:** Memorizza Email e Password (criptate con Argon2).
- **Utente:** Memorizza lo Username e i record (HighestScore, KillCount, DeathCount).

```
1 CREATE TABLE Credenziali (  
2     IdCredenziali INT PRIMARY KEY AUTO_INCREMENT,  
3     Email VARCHAR(255) NOT NULL UNIQUE,  
4     Password VARCHAR(255) NOT NULL  
5 );  
6  
7 CREATE TABLE Utente (  
8     IdUtente INT PRIMARY KEY AUTO_INCREMENT,  
9     Username VARCHAR(32) NOT NULL UNIQUE,  
10    HighestScore INT DEFAULT 0,  
11    KillCount INT DEFAULT 0,  
12    DeathCount INT DEFAULT 0,  
13    IdCredenziali INT,  
14    FOREIGN KEY (IdCredenziali) REFERENCES Credenziali(IdCredenziali)  
15 );
```

Listing 1: Schema DDL del Database

## 5 Protocollo di Comunicazione

Il protocollo utilizza messaggi in formato JSON per favorire la leggibilità durante la fase di sviluppo.

### 5.1 Handshake e Join

1. Il client invia un comando `join` via TCP.
2. Il server risponde con un `welcome` contenente un **Token univoco** e la porta UDP.
3. Il client invia un `hello` via UDP includendo il Token per autenticare la sessione senza i costi del TCP.

### 5.2 Snapshot del Mondo (UDP)

Ogni 33ms (30Hz), il server invia a ciascun giocatore uno "snapshot" contenente solo gli elementi visibili nel suo raggio d'azione (*Overfetch Radius*), riducendo il carico di banda rispetto all'invio dell'intera mappa.

## 6 Sviluppo Tecnico

### 6.1 Logica del Server

Il server esegue un *Game Loop* costante a 60 tick al secondo. In ogni tick: 1. Calcola il movimento dei giocatori in base al vettore di input. 2. Gestisce le collisioni (Player-Cibo,

Player-Player). 3. Gestisce lo "split" della massa e l'espulsione di pellet. 4. Aggiorna l'IA dei Bot.

## 6.2 Intelligenza Artificiale (Bot)

I bot implementano una logica decisionale complessa che include:

- **Evasione:** Fuga dai giocatori più grandi.
- **Inseguimento:** Caccia ai giocatori più piccoli.
- **Alleanza/Tradimento:** Meccanica avanzata dove i bot possono collaborare temporaneamente per poi tradirsi quando la massa accumulata diventa significativa.

## 6.3 Interpolazione Client-Side

Per eliminare gli scatti dovuti alla frequenza di rete (30Hz) rispetto al rendering (60Hz), il client implementa un sistema di **interpolazione lineare**. Le posizioni degli oggetti non vengono aggiornate istantaneamente, ma traslate fluidamente verso l'ultima posizione ricevuta dal server.

## 7 Conclusioni

Il progetto ha dimostrato come la scelta del protocollo di trasporto influenzi drasticamente l'esperienza utente in un'applicazione distribuita. L'integrazione tra Node.js e le tecnologie web moderne ha permesso di creare un sistema scalabile ed efficiente, gettando le basi per l'esplorazione di tecniche più avanzate come la *Client-Side Prediction* e la *Delta Compression*.